TP 4 — Versionner données et expérimentations avec DVC

Durée : 3 h · Partie 4 du cours · binôme de 2

Un modèle n'est reproductible que si le jeu de données qui l'a produit l'est aussi. Ce TP met les données sous contrôle de version, construit un pipeline dont DVC ne rejoue que les étapes impactees, et compare des expérimentations sans polluer l'historique Git. Version Windows : toutes les commandes sont écrites pour PowerShell 5.1 ou 7, livré avec Windows 10 et 11. Ouvrez « Windows PowerShell », jamais l'invite de commandes cmd.exe. Les blocs @' ... '@ sont des here-strings : le marqueur de fermeture '@ doit impérativement se trouver en début de ligne, sans espace devant, et le bloc entier se colle en une seule fois.

Objectifs pédagogiques

- Expliquer pourquoi Git seul ne convient pas au versioning des données
- Versionner un jeu de données et le publier sur un stockage distant
- Restaurer un état anterieur complet : code et données ensemble
- Décrire un pipeline dans dvc.yaml et observer le mécanisme de cache
- Comparer des expérimentations avec dvc exp et suivre les métriques

Prérequis

- TP 2 termine : les trois scripts du pipeline existent dans src/
- Depot Git initialise dans tp-mlops

Installation

Le stockage distant est ici un simple dossier local (C:\dvcstore). Les commandes sont identiques avec S3, Azure Blob ou MinIO : seule la ligne remote add change.

terminal

```
cd tp-mlops; .venv\Scripts\Activate.ps1
pip install "dvc[all]"                    # les crochets exigent les guillemets
dvc init                                  # initialise DVC dans le depot Git
New-Item -ItemType Directory -Force C:\dvcstore | Out-Null
dvc remote add -d local C:\dvcstore       # -d = stockage par defaut
git add .dvc .dvcignore; git commit -m 'chore: initialisation DVC'
```

RÉSULTAT ATTENDU

```
Initialized DVC repository.
Setting 'local' as a default remote.
```

Déroulement

Étape 1 — Mesurer le problème avant de le resoudre

On génère un fichier volumineux et on observe ce qu'il devient dans Git.

terminal

```
@'
import pandas as pd
df = pd.read_csv('data/transactions.csv')
gros = pd.concat([df] * 400, ignore_index=True)      # environ 2 millions de lignes
gros.to_csv('data/gros.csv', index=False)
'@ | python -
'{0:N1} Mo' -f ((Get-Item data/gros.csv).Length / 1MB)                      # taille du fichier
git add data/gros.csv; git commit -m 'test: gros fichier'
'{0:N1} Mo' -f ((Get-ChildItem .git -Recurse -File | Measure-Object Length -Sum).Sum / 1MB)                          # taille du depot apres commit
git reset --hard HEAD~1; rm -f data/gros.csv       # on annule ce test
```

TP 4 — Versionner données et expérimentations avec DVC

Durée : 3 h · Partie 4 du cours · binôme de 2

```
'{0:N1} Mo' -f ((Get-ChildItem .git -Recurse -File | Measure-Object Length -Sum).Sum / 1MB)                    # la taille NE redescend pas
```

RÉSULTAT ATTENDU

```
32M     data/gros.csv
35M     .git
35M     .git

Le fichier a disparu du repertoire de travail mais reste dans l'historique Git :
chaque collegue qui clone le depot le telechargera, pour toujours.
```

Étape 2 — Versionner le jeu de données avec DVC

terminal

```
dvc add data/transactions.csv             # sort le fichier de Git, cree le pointeur
Get-Content data/transactions.csv.dvc      # contenu du pointeur
(Get-Item data/transactions.csv.dvc).Length   # sa taille en octets
git add data/transactions.csv.dvc data/.gitignore
git commit -m 'data: jeu de donnees v1'
git tag v1                                 # etiquette lisible
dvc push                                   # envoie le contenu vers le stockage
```

RÉSULTAT ATTENDU

```
outs:
- md5: 6203c87193e8cda3ca48b453c49dbca0
  size: 80901
  hash: md5
  path: transactions.csv

97 octets
1 file pushed
```

Étape 3 — Publier une deuxième version des données

On simule une nouvelle collecte : les montants augmentent de 30 %.

terminal

```
@'
import pandas as pd
df = pd.read_csv('data/transactions.csv')
df['montant'] = (df['montant'] * 1.3).round(2)        # nouvelle collecte
df.to_csv('data/transactions.csv', index=False)
'@ | python -
dvc status                                 # DVC voit que le fichier suivi a change
dvc add data/transactions.csv
git add data/transactions.csv.dvc; git commit -m 'data: jeu de donnees v2'
git tag v2; dvc push
```

RÉSULTAT ATTENDU

```
data/transactions.csv.dvc:
■changed outs:
■■modified: data/transactions.csv

1 file pushed
```

TP 4 — Versionner données et expérimentations avec DVC

Durée : 3 h · Partie 4 du cours · binôme de 2

Étape 4 — Prouver le retour arriere complet

Les deux commandes vont toujours par paire : Git restaure le code et le pointeur, DVC restaure le contenu correspondant.

terminal

```
git checkout v1; dvc checkout              # retour a la version 1
Get-Content data/transactions.csv -TotalCount 2
git checkout main; dvc checkout            # retour a la version courante
Get-Content data/transactions.csv -TotalCount 2
```

RÉSULTAT ATTENDU

```
montant,anciennete,freq_24h,nuit,fraude
339.34,45,4,0,0

puis :
montant,anciennete,freq_24h,nuit,fraude
441.14,45,4,0,0

441.14 = 339.34 x 1.3 : le couple code + donnees est coherent.
```

Si ça coince : si votre branche s'appelle master, remplacez main par master.

Étape 5 — Décrire le pipeline dans dvc.yaml

On reprend les trois scripts du TP 2. DVC apprend leurs dépendances et leurs sorties.

dvc.yaml

```
@'
stages:
  preparer:
    cmd: python src/preparer.py
    deps: [src/preparer.py, data/transactions.csv]    # si l'un change, on rejoue
    params: [preparation.test_size]
    outs: [data/prepare]
  entrainer:
    cmd: python src/entrainer.py
    deps: [src/entrainer.py, data/prepare]
    params: [entrainement.n_estimators, entrainement.max_depth]
    outs: [models/model.pkl]
  evaluer:
    cmd: python src/evaluer.py
    deps: [src/evaluer.py, models/model.pkl, data/prepare]
    metrics:
      - reports/metrics.json:
          cache: false                                # suivi par Git, pas par DVC
'@ | Set-Content -Encoding UTF8 dvc.yaml
dvc repro                                  # premiere execution : tout tourne
dvc dag                                    # graphe des dependances
```

TP 4 — Versionner données et expérimentations avec DVC

Durée : 3 h · Partie 4 du cours · binôme de 2

RÉSULTAT ATTENDU

```
Running stage 'preparer':
Running stage 'entrainer':
Running stage 'evaluer':
Generating lock file 'dvc.lock'

dvc dag affiche les trois etapes reliees par des fleches.
```

Étape 6 — Observer le cache et le déclenchement cible

C'est le coeur du TP : DVC compare des empreintes et ne rejoue que le nécessaire.

terminal

```
dvc repro                                # rien n'a change
(Get-Content params.yaml) -replace 'max_depth: 12', 'max_depth: 6' | Set-Content params.yaml    # un seul hyperparametre
dvc repro                                # observez QUELLES etapes se relancent
dvc metrics show
```

RÉSULTAT ATTENDU

```
Stage 'preparer' didn't change, skipping
Stage 'entrainer' didn't change, skipping
Stage 'evaluer' didn't change, skipping
Data and pipelines are up to date.

puis, apres le changement de parametre :
Stage 'preparer' didn't change, skipping
Running stage 'entrainer':
Running stage 'evaluer':

Path                  f1
reports/metrics.json  0.8306
```

Étape 7 — Comparer des expérimentations

dvc exp permet de tester des réglages sans créer un commit par essai.

terminal

```
dvc exp run -S entrainement.max_depth=8
dvc exp run -S entrainement.max_depth=16
dvc exp run -S entrainement.n_estimators=600 -S entrainement.max_depth=20
dvc exp show --only-changed          # tableau comparatif des experimentations
dvc metrics diff HEAD                # ecart avec le dernier commit
```

RÉSULTAT ATTENDU

```
Un tableau listant chaque experimentation avec ses parametres et son f1.
Aucun commit Git n'a ete cree : l'historique reste propre.
```

Si ça coince : si l'affichage est trop large, ajoutez --drop 'Created' ou elargissez le terminal.

Étape 8 — Simuler le travail en équipe

Vérification finale : un collegue clone le depot et doit obtenir exactement vos données.

terminal

```
dvc push; git add .; git commit -m 'pipeline DVC complet'; git tag v3
```

TP 4 — Versionner données et expérimentations avec DVC

Durée : 3 h · Partie 4 du cours · binôme de 2

```
$projet = (Get-Location).Path
cd $env:TEMP; git clone $projet collegue; cd collegue
Get-ChildItem data                          # le CSV n'est pas la
dvc pull                                    # DVC le telecharge
(Get-Item data/transactions.csv).Length
Get-FileHash data/transactions.csv -Algorithm MD5   # meme empreinte que chez vous
```

RÉSULTAT ATTENDU

```
transactions.csv.dvc

puis, apres dvc pull :
A       data/transactions.csv
1 file added

L'empreinte md5 est identique a celle de votre poste.
```

Questions du compte rendu

1. Pourquoi la taille du depot .git ne redescend-elle pas après suppression du gros fichier ?
2. Que contient le fichier .dvc, et où se trouvent réellement les données ?
3. Un collegue lance git checkout v1 mais oublie dvc checkout. Que risque-t-il concretement ?
4. Vous modifiez uniquement max_depth. Pourquoi l'étape de préparation n'est-elle pas rejouee ?
5. DVC et MLflow se recouvrent-ils ? Precisez ce que chacun apporte et pourquoi on utilise les deux.

Livrables

- Le depot avec dvc.yaml, params.yaml, dvc.lock et les fichiers .dvc
- La sortie des deux dvc repro consecutifs (exécution puis cache)
- Le tableau dvc exp show de vos trois expérimentations
- La preuve du clone + dvc pull avec les empreintes md5 identiques
- Compte rendu avec les réponses aux questions

Barème

Critère	Points
Initialisation DVC, stockage distant, push et pull fonctionnels	4
Deux versions de données et retour arriere démontré	4
Pipeline dvc.yaml en trois étapes, correctement déclare	5
Démonstration du cache et du déclenchement cible	4
Expérimentations comparees et compte rendu	3